CS 162 Homework Test Completeness Audit
==========

Fallback PDF generated by build-pdfs.js because browser PDF printing was unavailable.

CS 162 Homework Test Completeness Audit
==========

Audit date: 2026-05-10

Scope

This report validates the homework documentation and the current testing state
of this repository. It is based on the public CS 162 homework pages, the files
in this repository, and a WSL Ubuntu execution pass from the repository root.

Official sources used:

- <https://cs162.org/static/hw/hw-intro/>
- <https://cs162.org/static/hw/hw-list/>
- <https://cs162.org/static/hw/hw-shell/>
- <https://cs162.org/static/hw/hw-http/>
- <https://cs162.org/static/hw/hw-http-rs/>
- <https://cs162.org/static/hw/hw-memory/>
- <https://cs162.org/static/hw/hw-map-reduce/>
- <https://cs162.org/static/hw/hw-map-reduce-rs/>

Environment validation

WSL Ubuntu is available and can execute the C and Pintos portions of the
repository. Rust homework testing is still blocked because Cargo is not
installed in the WSL image.

Attempted commands:

Check	Result	Impact
wsl -d Ubuntu -- bash -lc "uname -a"	Passed	WSL Ubuntu can be used for Linux-based validation.
command -v make	Passed: /usr/bin/make	C homework builds can be attempted.
command -v gcc	Passed: /usr/local/bin/gcc	C homework builds can be attempted.
cargo --version	Failed: cargo: command not found	Rust homework builds and tests cannot run until Rust is installed.
command -v qemu-system-i386	Passed: /usr/bin/qemu-system-i386	Pintos can run once its bundled pintos script is placed on PATH.
rg -n "TODO|todo!" ...	Passed	Static implementation gaps were found and recorded below.
rg -n "[\\p{Han}]" README.md hw-*/README.md docs/cs162-homework-guides	Passed	Repository homework documentation is English-only.
git diff --check	Passed	Markdown files do not contain whitespace errors.

Execution evidence summary

Homework	Command evidence	Runtime verdict

hw-intro	Top-level make builds map and limits; hw-intro/words make fails with -Werror=unused-but-set-variable for main.c:97 (infile).	Partially builds; word-count tests cannot run.
hw-list	make succeeds; ./pthread 2 runs as a smoke test.	Builds, but correctness is not proven.
hw-shell	make succeeds; bash test.sh fails because the script has CRLF line endings; a CRLF-normalized run still fails because pwd returns This shell doesn't know how to run programs.	Builds, but shell behavior fails the existing smoke test.
hw-http	make succeeds with warnings; a live httpserver --files www --port ... smoke test logs repeated Error accepting socket: Invalid argument, and curl cannot connect.	Builds, but the server fails a real socket smoke test.
hw-http-rs	cargo --version fails.	Not runnable in the current WSL image.
hw-memory/mm_alloc	make fails under -Werror because mm_malloc, mm_realloc, and mm_free parameters are unused.	Standalone allocator does not compile.
hw-memory/pintos/src/memory	make succeeds; make check fails without pintos on PATH; with PATH=../../utils:../utils:..., the active Pintos memory suite produced 26 result files: 1 pass and 25 failures.	Pintos builds, but memory tests are overwhelmingly failing.
hw-map-reduce	make fails at worker/../rpc/rpc.h:9:10: fatal error: rpc/rpc.h: No such file or directory.	C MapReduce does not compile in this WSL image.
hw-map-reduce-rs	cargo --version fails.	Not runnable in the current WSL image.

Static evidence summary

The repository is not ready to be called fully tested. Static inspection found 53 TODO or todo! markers across homework code and documentation. The most important markers are implementation placeholders, not just comments.

Observed implementation gap counts by homework:

Homework	Static TODO markers	Interpretation
hw-intro	0	No obvious placeholder markers, but words currently fails to compile under the provided flags.
hw-list	13	List and pthread word-count code still has visible TODO markers.
hw-shell	0	No TODO markers, but behavior is not implemented enough to pass the checked-in smoke test.
hw-http	15	The C HTTP server still has several part-level TODO markers and fails a socket smoke test.
hw-http-rs	6	The Rust HTTP server still has todo! placeholders in core modules.
hw-memory	3	The standalone allocator has unimplemented allocation functions; the Pintos memory suite largely fails.
hw-map-reduce	6	The C coordinator has visible TODO markers and the project does not compile in WSL without RPC header fixes.
hw-map-reduce-rs	10	The Rust coordinator and ignored test still contain placeholders.

Existing test inventory

Homework	Existing tests or harness	Completeness verdict
hw-intro	No checked-in automated harness found	Not complete
hw-list	No checked-in automated harness found	Not complete
hw-shell	test.sh checks one pwd case and one /bin/ls case, but currently fails	Not complete
hw-http	No checked-in automated HTTP harness found	Not complete
hw-http-rs	12 helper-level Rust tests in src/tests	Partial, not complete

hw-memory	mm_alloc/mm_test.c; 26 active Pintos memory tests in Make.tests; 36 memory .ck files total including the commented-out stack-growth group	Partial; available tests reveal major failures
hw-map-reduce	No checked-in automated cluster harness found	Not complete
hw-map-reduce-rs	One ignored test_wc containing todo!("Pick an input file")	Not complete

Homework-by-homework validation

HW 0: Intro

Observed WSL result:

- cd hw-intro && make built map and limits.
- cd hw-intro/words && make failed because main.c:97 defines infile, sets it, and never uses it while -Werror is enabled.

Documentation correctness check:

- The README points to the correct official HW 0 page and counting-words task.
- The listed files match the repository structure.
- The missing-test prompts cover word totals, frequency output, stdin, multiple files, invalid flags, limits, and map output shape.

Completeness verdict: not complete.

Required implementation fix before tests can run:

- Either use infile in the intended input-file path or remove the unused variable without changing the required command behavior.

Missing tests to add:

- Add words/tests/fixtures/basic.txt with repeated words and punctuation.
- Add a golden-output test for ./words -c tests/fixtures/basic.txt.
- Add a golden-output test for ./words -f tests/fixtures/basic.txt.
- Add stdin coverage with printf "one two one\n" | ./words -f.
- Add multi-file coverage that verifies counts are combined correctly.
- Add invalid-flag and missing-file tests that assert nonzero exit status and a useful diagnostic.
- Add smoke checks for ./limits and ./map so their output format is stable.

Minimum command set to run in Linux or WSL after the compile fix:

```
cd hw-intro
make
./limits
./map
cd words
make
./words -c tests/fixtures/basic.txt
./words -f tests/fixtures/basic.txt
```

HW 1: Lists

Observed WSL result:

- cd hw-list && make succeeded.
- ./pthread 2 ran, which only proves that one pthread smoke path starts.
- The smoke output alone does not prove that list.c, lwords, or pwords are correct.

Static evidence:

- word_count_l.c, word_count_p.c, and pwords.c still contain TODO markers.

Documentation correctness check:

- The README maps the official list and pwords tasks to list.c, word_count_l.c, word_count_p.c, and pwords.c.
- The recommended tests compare words, lwords, and pwords, which is the right local strategy for this homework.

Completeness verdict: not complete.

Missing tests to add:

- Add deterministic word-count fixtures for empty input, one word, repeated words, mixed case, punctuation, and multiple files.
- Compare lwords and pwords output against the reference words binary.
- Add a repeated race test that runs pwords at least 50 times on the same fixture and diffs every result against the reference output.
- Add a large-input test to exercise list insertion, lookup, sorting, and concurrent updates.
- Add sanitizer or Valgrind instructions for list memory ownership and thread lifetime bugs.

Minimum command set to run in Linux or WSL:

```
cd hw-list
make
./words tests/fixtures/single.txt > /tmp/ref.out
./lwords tests/fixtures/single.txt > /tmp/lwords.out
./pwords tests/fixtures/single.txt > /tmp/pwords.out
diff -u /tmp/ref.out /tmp/lwords.out
diff -u /tmp/ref.out /tmp/pwords.out
```

HW 2: Shell

Observed WSL result:

- cd hw-shell && make succeeded.
- bash test.sh failed first because the script has Windows CRLF endings.
- Running a CRLF-normalized version still failed the built-in pwd check.
- Manual probes such as printf "pwd\n" | ./shell and printf "/bin/echo hello\n" | ./shell returned
This shell doesn’t know how to run programs.

Documentation correctness check:

- The README covers the official shell surfaces: directory commands, foreground execution, redirection, pipes, Ctrl-D, Ctrl-C, and Ctrl-Z.
- The missing-test prompts include both deterministic command tests and environment-sensitive signal tests.

Completeness verdict: not complete.

Required implementation and harness fixes before broader testing:

- Convert test.sh to LF line endings or make the runner normalize line endings before execution.
- Implement built-ins and program execution enough for pwd, cd, and /bin/echo or /bin/ls to behave correctly.

Missing tests to add:

- Add table-driven tests for pwd, cd /, cd .., cd with no argument, and invalid cd targets.
- Add foreground execution tests for /bin/echo, /bin/true, /bin/false, and an invalid command.
- Add output-redirection tests that verify file content and overwrite behavior.
- Add pipe tests for echo hello | wc -w and a multi-stage pipeline.
- Add Ctrl-D behavior tests using stdin closure.
- Add Linux-only Ctrl-C and Ctrl-Z tests using a pseudo-terminal harness, with explicit timeouts.

Minimum command set to run in Linux or WSL:

```
cd hw-shell
make
bash test.sh
printf "cd /\npwd\n" | ./shell
printf "echo hello | wc -w\n" | ./shell
```

HW 3: HTTP Server in C

Observed WSL result:

- cd hw-http && make succeeded with warnings.
- A live server smoke test using ./httpserver --files www --port 18085 failed.
- The server log repeatedly printed Error accepting socket: Invalid argument.
- curl --noproxy 127.0.0.1 -i --max-time 3 http://127.0.0.1:18085/ could not connect.

Static evidence:

- httpserver.c contains part-level TODO markers for file serving, directory serving, proxying, and concurrency paths.
- No checked-in HTTP integration test script was found.

Documentation correctness check:

- The README maps the official socket and HTTP tasks to httpserver.c, libhttp.c, wq.c, and the server variants.
- The missing-test prompts require real socket tests, not just unit tests.

Completeness verdict: not complete.

Required implementation fix before deeper tests:

- Fix the accept loop so the server can accept a local TCP client without logging Invalid argument.
- Address compile warnings in paths that should return a value or use required parameters.

Missing tests to add:

- Add tests/http_smoke.sh that starts a server on a random available port, waits for readiness, sends requests, and always kills the server on exit.
- Run the same smoke harness against httpserver, forkserver, threadserver, and poolserver.
- Add 200, 404, and malformed-request tests.
- Add header tests for Content-Length, connection behavior, and MIME type where required.
- Add directory listing and index.html tests if those are part of the target assignment version.
- Add path traversal tests such as GET /../secret.
- Add proxy tests with a small local upstream server.
- Add concurrency tests that issue many simultaneous requests and compare every response body.

Minimum command set to run in Linux or WSL after the accept fix:

```
cd hw-http
make
./httpserver --files www --port 8000
curl --noproxy 127.0.0.1 -i http://127.0.0.1:8000/
curl --noproxy 127.0.0.1 -i http://127.0.0.1:8000/not-found
```

HW 3: HTTP Server in Rust

Observed WSL result:

- cargo --version failed with cargo: command not found.
- Rust build and test commands were not runnable in the current WSL image.

Static evidence:

- src/http.rs, src/server.rs, and src/stats.rs contain todo! placeholders.
- src/tests contains 12 helper-level tests.
- No integration test starts the real server and sends TCP requests.

Documentation correctness check:

- The README correctly separates helper-level testing from socket-level integration testing.
- The missing-test prompts call for parser tests, real GET tests, directory tests, stats integration checks, and concurrent client coverage.

Completeness verdict: partial, not complete.

Required environment fix before testing:

- Install Rust and Cargo in WSL, then run the command set below.

Missing tests to add:

- Keep the current helper tests.
- Add request parser tests for valid requests, bad methods, bad paths, missing headers, partial reads, and malformed bytes.
- Add tests/server_integration.rs that starts the server on an ephemeral port and sends real TCP requests.
- Add file-serving tests for success, not found, directory handling, and path traversal.
- Add stats tests that make real requests and verify the stats endpoint.
- Add concurrent-client tests with timeouts.

Minimum command set to run in Linux, WSL, or a Rust environment:

```
cd hw-http-rs
cargo test
cargo test --test server_integration
cargo run -- --files www --port 8000
```

HW 4: Memory

Observed WSL result:

- cd hw-memory/mm_alloc && make failed because mm_malloc, mm_realloc, and mm_free leave parameters unused under -Werror.
- cd hw-memory/pintos/src/memory && make succeeded.
- make check first failed because pintos was not on PATH.
- With PATH=../../utils:../utils:..., the active Pintos memory suite produced 26 result files before manual cleanup of the long-running command.
- Only malloc-null passed. The other 25 active results failed, including sbrk-small, sbrk-zero, sbrk-many, malloc-simple, malloc-free, malloc-fit, malloc-fail, malloc-merge-*, and realloc-*.

Static evidence:

- mm_alloc/mm_alloc.c contains TODO markers for malloc, realloc, and free.
- pintos/src/tests/memory/Make.tests enables 26 active memory tests.
- The directory contains 36 memory .ck files total; the stack-growth group is present but commented out for this assignment version.

Documentation correctness check:

- The README correctly distinguishes the standalone allocator from the Pintos memory suite.
- It avoids claiming active calloc coverage where the current active memory test list does not show explicit calloc tests.

Completeness verdict: partial, not complete.

Required implementation and environment fixes before full validation:

- Implement mm_malloc, mm_realloc, and mm_free enough for mm_alloc to

compile.
- Run Pintos checks with the bundled utilities on PATH.
- Investigate the sbrk failures first; many allocator tests fail because the user heap contract is not working.

Missing tests to add:

- Expand standalone allocator tests for alignment, zero-size allocation, allocation reuse, splitting, coalescing, realloc growth, realloc shrink, realloc-to-null behavior, free-null behavior, double-free policy, and fragmentation stress.
- Add a deterministic standalone stress test that allocates and frees many differently sized blocks.
- Add a test that writes through every allocated byte to catch invalid returned pointers.
- Add a documented Pintos command that includes the required PATH setup.
- Decide whether stack-growth tests are required for the target assignment version before enabling them.

Minimum command set to run in Linux or the course VM:

```
cd hw-memory/mm_alloc
make
./mm_test

cd ../pintos/src/memory
make
env PATH=../../utils:../utils:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
make check
```

HW 5: MapReduce in C

Observed WSL result:

- cd hw-map-reduce && make failed while compiling worker/worker.o.
- The immediate error was worker/../rpc/rpc.h:9:10: fatal error: rpc/rpc.h: No such file or directory.

Static evidence:

- The C coordinator contains TODO markers.
- No checked-in integration harness was found.

Documentation correctness check:

- The README maps the official job-submission and cluster behavior to the RPC, coordinator, worker, client, and app directories.
- The missing-test prompts require a real cluster test with cleanup and golden output comparison.

Completeness verdict: not complete.

Required environment or build fix before tests can run:

- Install the required SunRPC/TIRPC development headers in WSL, or adjust the include path and Makefile so rpc/rpc.h resolves consistently.

Missing tests to add:

- Add tests/run_cluster_test.sh that starts the coordinator and workers, submits a job, processes output, validates output, and kills all child processes on exit.
- Add wc, grep, and vertex_degree golden-output fixtures.
- Add a no-input or empty-input job test.
- Add repeated job-submission tests to verify job IDs and output directories.
- Add worker-failure retry coverage by killing a worker mid-task.
- Add timeout handling so a hung coordinator or worker fails the test instead of blocking forever.

Minimum command set to run in Linux or WSL after RPC headers are fixed:

```
cd hw-map-reduce
make
./bin/mr-coordinator
./bin/mr-worker
./bin/mr-client submit -a wc -o /tmp/hw-map-reduce-out -w data/gutenberg/p.txt
./bin/mr-client process -a wc -o /tmp/hw-map-reduce-out
```

HW 5: MapReduce in Rust

Observed WSL result:

- cargo --version failed with cargo: command not found.
- Rust build and test commands were not runnable in the current WSL image.

Static evidence:

- src/coordinator/mod.rs still contains job-submission TODOs.
- src/tests/mod.rs::test_wc is ignored and contains todo!("Pick an input file").

Documentation correctness check:

- The README covers worker registration, job submission, task assignment, completion, heartbeat, retry, and end-to-end output validation.
- The missing-test prompts ask for both state-machine unit tests and async integration tests.

Completeness verdict: not complete.

Required environment fix before testing:

- Install Rust and Cargo in WSL, then run the command set below.

Missing tests to add:

- Finish test_wc with a real fixture and expected output.
- Add worker-registration tests for unique worker IDs and duplicate registrations.
- Add job-submission tests for valid apps, invalid apps, missing input files, and output-directory collisions.
- Add task-assignment tests for map tasks, reduce tasks, and no-work responses.

- Add completion tests for successful tasks, failed tasks, duplicate completions, and stale worker reports.
- Add heartbeat and retry tests with controlled timeouts.
- Add wc, grep, and vertex_degree async integration tests that start the coordinator and workers and compare final outputs to golden files.

Minimum command set to run in Linux, WSL, or a Rust environment:

```
cd hw-map-reduce-rs
cargo test
cargo test test_wc -- --ignored
```

Overall verdict

The homework documentation is now stronger than a plain official assignment page because it is tied directly to this repository and its current testing state. The WSL validation pass also shows that the assignments themselves are not fully tested and, in several cases, are not yet passing basic build or smoke-test gates.

Readiness by homework:

Homework	Documentation readiness	Test completeness	Current execution state
HW 0 Intro	Good	Not complete	Partial C build; words fails to compile
HW 1 Lists	Good	Not complete	Builds; only pthread smoke was run
HW 2 Shell	Good	Not complete	Builds; checked-in smoke test fails
HW 3 HTTP C	Good	Not complete	Builds; socket smoke test fails
HW 3 HTTP Rust	Good	Partial, not complete	Blocked by missing Cargo
HW 4 Memory	Good	Partial, not complete	mm_alloc fails to compile; Pintos suite mostly fails
HW 5 MapReduce C	Good	Not complete	Build fails on missing RPC header
HW 5 MapReduce Rust	Good	Not complete	Blocked by missing Cargo

Definition of done for future work

Each homework should be considered well tested only after all of the following are true:

1. The homework builds in the expected Linux, WSL, or course VM environment.
2. The documented smoke commands pass.
3. The repository contains deterministic automated tests for the main required behavior.
4. Edge cases and failure modes listed in each README are covered.
5. Long-running or environment-sensitive tests have timeouts.
6. The test commands are documented and reproducible from a fresh checkout.
7. Any skipped or ignored test is explained in the README or audit report.